1 What is SQL

```
- SQL stands for Structured Query Language
- You use it to read and manage data in relational databases
- Used in MySQL, PostgreSQL, SQL Server, Oracle
```

2 What is an RDBMS

- Relational Database Management System
- Stores data in tables with rows and columns
- Uses keys to link tables
- Example. Customer table linked to Orders table using customer_id

3 What is a table

- Structured storage for data
- Rows are records
- Columns are attributes
- Example. One row equals one customer

4 What is a primary key

- Uniquely identifies each row
- Cannot be NULL
- No duplicate values
- Example. user_id in users table

5 What is a foreign key

- Links one table to another
- Refers to a primary key in another table
- Allows duplicate values
- Example. user_id in orders table

6 Difference between primary key and foreign key

- Primary key ensures uniqueness
- Foreign key ensures relationship
- One table can have one primary key
- One table can have multiple foreign keys

7 What is NULL

- Represents missing or unknown value
- Not equal to zero or empty string
- Use IS NULL or IS NOT NULL to check

8 What are constraints

- Rules applied on columns
- Maintain data quality
- Common constraints – NOT NULL – UNIQUE – PRIMARY KEY – FOREIGN KEY – CHECK

9 What are data types

- Define type of data stored
- Common types – INT for numbers – VARCHAR for text – DATE for dates – FLOAT or DECIMAL for decimals

10 Interview tip you must remember

- Always explain with a small example
- Speak logic before syntax
- Keep answers short and direct

Double Tap ♥ For More

☑ *Core SQL Interview Questions With Answers - Part 2* 💻

1. What is SELECT

- Retrieves specific data from tables
- Choose columns with * for all
- Example: SELECT name, age FROM users;

1. What does WHERE do

- Filters rows based on conditions
- Applied after FROM clause
- Example: SELECT * FROM users WHERE age > 25 [1]

1. What is ORDER BY

- Sorts result set by column(s)
- ASC (default) or DESC
- Example: SELECT * FROM users ORDER BY age DESC;

1. What is GROUP BY

- Groups rows with same values
- Used with aggregate functions like COUNT, SUM
- Example: SELECT department, COUNT(*) FROM employees GROUP BY department [2]

1. Difference between WHERE and HAVING

- WHERE filters rows before grouping
- HAVING filters groups after GROUP BY
- WHERE can't use aggregates; HAVING can

1. What is a JOIN

- Combines rows from two or more tables

- Based on related columns
- INNER JOIN returns matching rows only

1. Types of JOINs

- INNER JOIN: matching rows
- LEFT JOIN: all from left + matches from right
- RIGHT JOIN: all from right + matches from left
- FULL OUTER JOIN: all from both

1. What are aggregate functions

- Perform calculations on groups
- COUNT, SUM, AVG, MIN, MAX
- Example: SELECT AVG(salary) FROM employees

1. What is a subquery

- Query inside another query
- Used in SELECT, WHERE, FROM
- Example: SELECT * FROM users WHERE age > (SELECT AVG(age) FROM users)

1. Interview tip you must remember

- Explain query execution order: FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
- Use simple examples with 2-3 rows
- Practice on sample datasets like employees/orders

Double Tap 🤝 *For Part 3*

☑️ *Advanced SQL Queries Interview Questions With Answers* 💻

1. *What is a JOIN?*

- JOIN combines rows from two or more tables based on a related column.
- Common types: `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, `FULL OUTER JOIN`.
  Example:

```
SELECT e.name, d.dept_name
FROM employees e
JOIN departments d ON e.dept_id = d.id;
```

2. *What is the difference between INNER JOIN and LEFT JOIN?*

- `INNER JOIN` returns only matching rows in both tables.
- `LEFT JOIN` returns all rows from the left table plus matching rows from the right; unmatched right-side columns are `NULL`.
  Example:

```
SELECT e.name, o.order_amount
FROM employees e
LEFT JOIN orders o ON e.id = o.emp_id;
```

3. *What is a subquery?*

- A subquery is a `SELECT` inside another query (in `WHERE`, `FROM`, or `SELECT`).
- Can be correlated (uses outer-query columns) or un-correlated (independent).
  Example (WHERE):

```
SELECT * FROM employees
WHERE salary > (
    SELECT AVG(salary) FROM employees
);
```

4. *What is the difference between correlated and nested (non-correlated) subquery?*

- *Correlated*: Inner query references outer-query columns and runs once per outer row.
- *Nested*: Inner query is independent; runs once and its result is reused.
  Example (correlated):

```
SELECT e1.name, e1.salary
FROM employees e1
WHERE e1.salary > (
    SELECT AVG(e2.salary)
    FROM employees e2
    WHERE e2.dept_id = e1.dept_id
);
```

5. *What is the difference between subquery and JOIN?*

- *JOIN* is usually faster and more readable for combining tables.
- *Subquery* is useful when you need aggregation or logic before joining, or when the join condition is complex.
  Use joins for simple look-ups; use subqueries when you need pre-computed aggregates or filters.

6. *What is a CTE (Common Table Expression)?*

- CTE is a named temporary result set defined with `WITH` clause.
- Improves readability and allows recursion.
  Example:

```
WITH high_salaries AS (
    SELECT *
    FROM employees
```

```
    WHERE salary > 80000
)
SELECT name, salary
FROM high_salaries
WHERE dept_id = 101;
```

7. *What are window functions?*

- Window functions compute values across a "window" of rows without reducing the row count.
- Use `OVER(...)` after an aggregate or ranking function.
  Example (running total):

```
SELECT customer_id, order_date, amount,
       SUM(amount) OVER (
           PARTITION BY customer_id
           ORDER BY order_date
       ) AS running_total
FROM orders;
```

8. *What is the difference between GROUP BY and window functions?*

- `GROUP BY` collapses rows into groups; only grouped/aggregated columns can appear.
- Window functions keep all rows and add computed values using `PARTITION BY` and `ORDER BY`.
  Use `GROUP BY` for final aggregates; use window functions for per-row analytics (rank, running total, lag/lead).

9. *What are ranking functions: ROW_NUMBER, RANK, DENSE_RANK?*

- `ROW_NUMBER()`: 1,2,3... no ties.
- `RANK()`: skips ranks on ties (1,1,3).
- `DENSE_RANK()`: no gaps (1,1,2).
  Example (top 1 order per customer):

```
WITH ranked_orders AS (
    SELECT *,
           ROW_NUMBER() OVER (
               PARTITION BY customer_id
               ORDER BY order_date DESC
           ) AS rn
    FROM orders
)
SELECT * FROM ranked_orders WHERE rn = 1;
```

10. *What is the difference between UNION and UNION ALL?*

- `UNION` combines two result sets and removes duplicates.

- `UNION ALL` combines and keeps all rows, faster because no deduplication.
  Both must have same number of columns and compatible data types.

11. *What is the difference between DISTINCT and GROUP BY?*

- `DISTINCT` removes duplicate rows from the final result.
- `GROUP BY` groups rows for aggregation; often used with `COUNT`, `SUM`, etc.
  Example:

```
SELECT dept_id, COUNT(DISTINCT employee_id)
FROM employees
GROUP BY dept_id;
```

12. *How do you handle NULLs in SQL?*

- Use `IS NULL` / `IS NOT NULL` for comparisons (never `= NULL`).
- `COALESCE(expression, default_value)` returns first non-null value.
  Example:

```
SELECT name, COALESCE(email, 'No email') AS email
FROM customers;
```

13. *What is an index and how does it help?*

- An index is a data structure (often B-tree) that speeds up data retrieval.
- Helps queries with `WHERE`, `JOIN`, `ORDER BY`, but can slow down `INSERT`/`UPDATE`/`DELETE`.
  Typical use: add index on `customer_id` when you often filter by it.

14. *How do you optimize a slow SQL query?*
  Common techniques:

- Add appropriate indexes on columns used in `WHERE`, `JOIN`, and `ORDER BY`.
- Avoid `SELECT *`; fetch only needed columns.
- Prefer `INNER JOIN` over subqueries where possible.
- Use `LIMIT` / pagination for large results.
- Avoid functions on indexed columns in `WHERE` (e.g., `WHERE YEAR(date_col) = 2023` harms index usage).

15. *What is the basic execution order in SQL?*
  Interview-friendly:
  `FROM` → `WHERE` → `GROUP BY` → `HAVING` → `SELECT` → `ORDER BY` → `LIMIT` / `OFFSET`.
  Mention this when explaining why you put filters in `WHERE` vs `HAVING`.

Quick advanced-level interview advice

- For *top-N / latest-record* problems, think `ROW_NUMBER()` + `PARTITION BY`.
- For *running totals / %-of-total*, default to `SUM(...) OVER(...)` window functions instead of self-joins.

- For *complex logic*, use CTEs to break the query into clear steps and talk through each CTE.
- Always mention *indexes* and *filtering on indexed columns* when discussing performance.

Double Tap ♥ *For More*

☑ *Core SQL Interview Questions With Answers - Part 3* 💻

21 What is COUNT function

- Counts number of rows or non-NULL values
- COUNT(*) counts all rows including NULLs
- Example: SELECT COUNT(*) FROM orders WHERE status = 'shipped';

22 What is SUM function

- Adds up numeric values in column
- Ignores NULL values
- Example: SELECT SUM(amount) FROM orders;

23 What is AVG function

- Calculates average of numeric column
- Ignores NULL values
- Example: SELECT AVG(salary) FROM employees WHERE department = 'IT';

24 What is MIN and MAX

- Finds smallest/largest value
- Works on numbers, dates, strings
- Example: SELECT MIN(order_date), MAX(order_date) FROM orders;

25 What are window functions

- Perform calculations across row sets
- ROW_NUMBER(), RANK(), DENSE_RANK()
- Example: ROW_NUMBER() OVER (ORDER BY salary DESC);

26 What does DISTINCT do

- Removes duplicate rows from result
- Use in SELECT or with COUNT
- Example: SELECT DISTINCT department FROM employees;

27 What is LIMIT

- Restricts number of rows returned
- TOP in SQL Server, ROWNUM in Oracle
- Example: SELECT * FROM employees ORDER BY salary DESC LIMIT 5;

28 What is OFFSET

- Skips specified number of rows
- Used with LIMIT for pagination

- Example: SELECT * FROM employees LIMIT 10 OFFSET 20;

29 What are indexes

- Speed up data retrieval
- Like book index for fast lookup
- CREATE INDEX idx_name ON table(column);

30 Interview tip you must remember

- Always mention execution order impact on performance
- Practice explaining JOIN visuals (Venn diagrams)
- Know when to use WHERE vs HAVING vs window functions

Double Tap ♥ For Part 4

☑ *Core SQL Interview Questions With Answers - Part 4* 💻

31 What are SQL Views

- Virtual tables based on query results
- Simplify complex queries
- Example: CREATE VIEW high_earners AS SELECT * FROM employees WHERE salary > 50000;

32 What is a Stored Procedure

- Pre-compiled SQL code block
- Reusable, reduces network traffic
- Example: CALL GetEmployeeReport(1001);

33 What are SQL Triggers

- Auto-execute on events (INSERT, UPDATE, DELETE)
- Enforce business rules automatically
- Example: Trigger to log salary changes;

34 Difference between DELETE and TRUNCATE

- DELETE removes specific rows, logs each
- TRUNCATE removes all rows, faster, no logging
- DELETE can use WHERE; TRUNCATE cannot;

35 What is ACID property

- Atomicity: All or nothing
- Consistency: Valid state always
- Isolation: Transactions independent
- Durability: Committed data persists;

36 What is Normalization

- Organize data to reduce redundancy
- 1NF, 2NF, 3NF levels

- Example: Split customer and address tables;

37 What is Denormalization

- Add redundancy for read performance
- Used in data warehouses
- Trade-off: faster queries vs storage;

38 What are Transactions

- Group of operations as single unit
- BEGIN TRANSACTION, COMMIT, ROLLBACK
- Example: Bank transfer (deduct + credit);

39 What is an Index Scan vs Seek

- Scan: reads entire index/table
- Seek: directly finds rows (faster)
- Use covering indexes for best performance;

40 Interview tip you must remember

- Draw ER diagrams for relationships
- Always mention when to use views vs temp tables
- Practice EXPLAIN ANALYZE for query optimization

Double Tap ❤️ For Part 5 ☑️ Core SQL Interview Questions With Answers - Part 5 💻

41 What are UNION and UNION ALL

- Combines results from multiple SELECTs
- UNION removes duplicates; UNION ALL keeps them
- Example: SELECT name FROM customers UNION SELECT name FROM suppliers;

42 What is CASE statement

- Conditional logic like if-else
- Multiple WHEN clauses possible
- Example: SELECT name, CASE WHEN salary > 50000 THEN 'High' ELSE 'Low' END AS level FROM employees;

43 What are CTEs (Common Table Expressions)

- Temporary result set, readable alternative to subqueries
- Use WITH clause
- Example: WITH high_earners AS (SELECT * FROM employees WHERE salary > 50000) SELECT * FROM high_earners;

44 Difference between CTE and Subquery

- CTE reusable within query, more readable
- Subquery embedded, single-use only

- CTEs support recursion;

45 What is a Self Join

- Join table with itself
- Useful for hierarchical data (manager-employee)
- Example: SELECT e.name, m.name AS manager FROM employees e JOIN employees m ON e.manager_id = m.id;

46 What is LEFT JOIN vs INNER JOIN

- INNER: only matching rows
- LEFT: all left table + matching right (NULLs for no match)
- Visualize with Venn diagrams;

47 What is a Correlated Subquery

- Subquery references outer query columns
- Executes once per outer row (slower)
- Example: SELECT name FROM employees e WHERE salary > (SELECT AVG(salary) FROM employees WHERE department = e.department);

48 What does EXISTS do

- Tests if subquery returns rows
- Faster than IN for large datasets
- Example: SELECT * FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);

49 What is SQL Injection

- Attack injecting malicious SQL via input
- Prevent with parameterized queries/prepared statements
- Example: Never concatenate user input directly;

50 Interview tip you must remember

- Always explain performance implications first
- Use EXPLAIN PLAN to analyze queries
- Practice real datasets (employees, orders, products)
- Know your DBMS specifics (MySQL vs PostgreSQL)

Double Tap 💖 For Part 6 ☑ Core SQL Interview Questions With Answers - Part 6 💻

51 What is INSERT statement

- Adds new rows to table
- Specify columns or use defaults
- Example: INSERT INTO employees (name, salary) VALUES ('Amit', 45000);

52 What is UPDATE statement

- Modifies existing rows
- Always use WHERE to avoid updating all rows
- Example: UPDATE employees SET salary = salary * 1.1 WHERE department = 'IT';

53 What is DELETE statement

- Removes rows from table
- WHERE clause critical to avoid data loss
- Example: DELETE FROM orders WHERE status = 'cancelled';

54 What are DML vs DDL

- DML: Data Manipulation (INSERT, UPDATE, DELETE, SELECT)
- DDL: Data Definition (CREATE, ALTER, DROP, TRUNCATE)
- DML changes data; DDL changes structure;

55 What is a Clustered Index

- Determines physical order of data in table
- Only one per table (usually primary key)
- Faster for range scans and sorted queries;

56 What is a Non-Clustered Index

- Separate structure pointing to data
- Multiple allowed per table
- Good for exact match lookups;

57 What is Query Optimization

- Finding most efficient execution plan
- Use EXPLAIN/EXPLAIN ANALYZE
- Indexes, statistics, rewrite queries;

58 Difference between OLTP and OLAP

- OLTP: transactional (fast writes, small reads)
- OLAP: analytical (fast reads, complex queries)
- OLTP normalized; OLAP denormalized;

59 What is a Pivot Table in SQL

- Transforms rows to columns dynamically
- Use CASE with aggregates
- Example: Sales by month as columns;

60 Interview tip you must remember

- Always backup before DDL operations
- Test UPDATE/DELETE on small datasets first
- Know index maintenance costs
- Practice window functions on real data

Double Tap ♥ For Part 7

☑ *Core SQL Interview Questions With Answers - Part 7* 💻

61 What is RANK() vs DENSE_RANK()

- RANK() skips numbers after ties (1,2,2,4)
- DENSE_RANK() no skips (1,2,2,3)
- Example: RANK() OVER (ORDER BY salary DESC)

62 What is ROW_NUMBER()

- Assigns sequential numbers to rows
- Unique even with ties (1,2,3,4)
- Example: ROW_NUMBER() OVER (ORDER BY name)

63 What is LAG() function

- Access previous row value
- Great for calculating differences
- Example: LAG(salary) OVER (ORDER BY hire_date)

64 What is LEAD() function

- Access next row value
- Useful for forecasting trends
- Example: LEAD(salary) OVER (ORDER BY hire_date)

65 What is PARTITION BY

- Divides rows into groups within window
- Resets ranking per partition
- Example: RANK() OVER (PARTITION BY dept ORDER BY salary)

66 What is recursive CTE

- CTE that references itself
- Solves hierarchical queries
- Example: Employee-manager hierarchy tree

67 What does EXPLAIN do

- Shows query execution plan
- Identifies slow operations
- Use EXPLAIN ANALYZE for timing

68 What is Covering Index

- Index contains all SELECT columns
- Avoids table lookup (bookmark lookup)
- CREATE INDEX idx_cover ON table(col1,col2)

69 What is Composite Index

- Index on multiple columns
- Order matters (leftmost first)
- Good for frequent WHERE combinations

70 Interview tip you must remember

- Draw window function execution flow
- Practice EXPLAIN on slow queries first
- Know when indexes hurt performance (small tables, many writes)
- Always test window functions with sample data

Double Tap ♥ For Part 8

☑ *Core SQL Interview Questions With Answers - Part 8* 💻

71 What is SQL Deadlock

- Two transactions block each other
- Both wait for locks held by other
- Solution: shorter transactions, proper indexing

72 What are Table Locks vs Row Locks

- Table lock: entire table blocked
- Row lock: only affected rows blocked
- Row locks more granular, better concurrency

73 What is Isolation Level

- Controls transaction visibility
- READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SERIALIZABLE
- Balance consistency vs performance

74 What is a Temp Table

- Session-specific table for intermediate results
- #temp in SQL Server, CREATE TEMP TABLE in PostgreSQL
- Faster than CTE for complex multi-step queries

75 What is Table Variable

- Memory-based, scoped to batch
- @table in SQL Server
- No statistics, good for small datasets

76 What does COALESCE do

- Returns first non-NULL value
- Alternative to ISNULL
- Example: COALESCE(phone1, phone2, 'No phone')

77 What is NULLIF

- Returns NULL if equal, else first arg
- Divides safely: amount/NULLIF(count,0)
- Prevents divide by zero errors

78 What are PIVOT and UNPIVOT

- PIVOT: rows to columns
- UNPIVOT: columns to rows
- Dynamic crosstabs without CASE statements

79 What is MERGE statement

- UPSERT operation (update if exists, insert if not)
- Single statement for insert/update/delete
- Example: MERGE target USING source ON match_condition

80 Interview tip you must remember

- Always ask about data volume in performance questions
- Mention transaction isolation for concurrency scenarios
- Practice deadlock scenarios with 2+ transactions
- Know your DBMS lock escalation rules

Double Tap *For Part 9* ☑ *Core SQL Interview Questions With Answers - Part 9*

81 What is Full Outer Join

- All rows from both tables
- NULLs where no match exists
- LEFT + RIGHT JOIN UNION equivalent

82 What is Cross Join

- Cartesian product (every row x every row)
- Rarely used intentionally
- Example: 5 customers x 3 products = 15 rows

83 What are Date Functions

- DATEADD, DATEDIFF, DATEPART
- CURRENT_DATE, NOW(), YEAR(), MONTH()
- Example: DATEDIFF(day, order_date, GETDATE())

84 What is String Functions

- CONCAT, SUBSTRING, LENGTH, UPPER, LOWER
- LEFT, RIGHT, TRIM, REPLACE
- Example: CONCAT(first_name, ' ', last_name)

85 What does LIKE do

- Pattern matching with % and _

- % matches any characters, _ matches one
- Example: name LIKE 'A%' finds all starting with A

86 What are Common Table Indexes

- Speed up repeated column access
- INCLUDE columns for covering indexes
- Avoid SELECT * in WHERE clauses

87 What is Query Hint

- Force optimizer choices
- USE INDEX, FORCE INDEX, MAX_ROWS
- Use sparingly, test performance impact

88 What is Materialized View

- Physical storage of query result
- Faster than regular views
- REFRESH MATERIALIZED VIEW periodically

89 What is Partitioned Table

- Split large table across files/partitions
- By date, range, hash, list
- Improves query pruning performance

90 Interview tip you must remember

- Always qualify columns (table.column)
- Use table aliases consistently
- Test edge cases (NULLs, empty tables)
- Mention data types in function examples

Double Tap ❤️ For Part 10

☑ *Core SQL Interview Questions with Answers - Part 10* 💻

91 What are the 4 Main JOIN Types

- INNER JOIN: matching rows only
- LEFT JOIN: all left + matching right (NULLs for no match)
- RIGHT JOIN: all right + matching left
- FULL OUTER JOIN: all rows from both tables

92 Difference Between INNER vs LEFT JOIN

- INNER: only where join condition matches
- LEFT: all left table rows, NULLs when no right match
- INNER faster, LEFT preserves all source data

93 What is CROSS JOIN

- Cartesian product (every row × every row)
- No ON condition needed
- Rarely used: 100 customers × 5 products = 500 rows

94 What is SELF JOIN

- Table joined with itself
- Uses table aliases (e.emp_id = m.emp_id)
- Perfect for hierarchies (employee-manager)

95 What is Window Function Syntax

- Function() OVER (PARTITION BY col ORDER BY col ROWS/RANGE frame)
- PARTITION BY: groups, ORDER BY: sequence
- ROWS UNBOUNDED PRECEDING default frame

96 ROW_NUMBER() vs RANK() vs DENSE_RANK()

- ROW_NUMBER(): 1,2,3,4 (unique)
- RANK(): 1,2,2,4 (ties share, skips)
- DENSE_RANK(): 1,2,2,3 (ties share, no skips)

97 What does PARTITION BY do

- Creates separate windows within result set
- Resets numbering/aggregates per partition
- Like GROUP BY but keeps all detail rows

98 What is LAG() / LEAD() Theory

- LAG: previous row value, LEAD: next row value
- Default offset=1, can specify columns
- Perfect for growth rates, trends

99 What are Aggregate Window Functions

- SUM(), COUNT(), AVG() OVER() windows
- Running totals: SUM() OVER (ORDER BY date ROWS UNBOUNDED PRECEDING)
- Moving averages: AVG() OVER (ROWS 7 PRECEDING)

100 Interview tip you must remember

- Draw Venn diagrams for JOIN questions
- Always specify PARTITION BY + ORDER BY for windows
- Test window functions with 5-10 row datasets
- Execution: Window functions after GROUP BY, before ORDER BY

Double Tap ♥ For More!